Bug Bounty Foundations: Scope, Evidence, Reports

A friendly field guide to learning web security responsibly and preparing clear vulnerability reports

1. What is bug bounty work?

A bug bounty program invites researchers to examine specified assets under published rules. A useful researcher is careful, methodical, respectful of users, and honest about what was actually demonstrated. Participation or payment is never guaranteed.

FIRST RULE Written program scope and rules come before tools. If an asset, test, or action is not clearly allowed, stop and ask the program owner.

2. Five checks before testing

Check	Ask yourself
Asset scope	Is this exact domain, app, API, or repository listed as in scope?
Allowed methods	Are automation, scanners, authenticated testing, and particular test types permitted?
Boundaries	What is excluded, rate-limited, or prohibited?
Data safety	Could this expose, change, or delete someone else's information? Do not proceed if so.
Disclosure	Where must findings be reported, and what sharing limits apply?

3. A safe beginner practice path

1. Learn HTTP requests, responses, cookies, sessions, authentication, and access control using diagrams and official documentation.
2. Complete a beginner lab specifically provided by PortSwigger Web Security Academy. Stay within that lab's boundaries.
3. For local practice, run an intentionally vulnerable app such as OWASP Juice Shop only on your own machine or isolated training network; do not expose it publicly.
4. Use browser developer tools to observe normal requests in that training application before making any test changes.
5. Write down the expected behaviour, observed behaviour, and the smallest reproducible proof.
6. Practise remediation thinking: explain how a developer could prevent or limit the issue.

4. Example: an access-control question

Friendly example: in a deliberately designed training lab, a page appears to show a profile belonging to the currently signed-in demo user. The learning question is whether the application consistently checks who is allowed to view a record. Use only the lab's provided accounts and instructions; never try another person's real account or data.

Report section	What makes it useful
Title	Name the behaviour and affected feature in plain language.
Scope	Identify the permitted training target or in-scope asset.
Summary	Explain the access-control expectation and what differed.
Reproduction	Give minimal, repeatable steps using only permitted test accounts.
Impact	Describe realistic consequences without exaggerating.
Evidence	Include redacted screenshots or sample responses with secrets removed.
Fix idea	Recommend server-side authorisation checks and appropriate tests.

5. Report template

```
Title: [behaviour] in [feature]
Asset / lab: [exact authorised scope]
Summary: [one or two plain-language sentences]
Steps: [minimal repeatable steps in the lab]
Expected: [intended safe behaviour]
Observed: [what actually happened]
Impact: [who/what could be affected, evidence-based]
Evidence: [redacted screenshot/request/response]
Suggested fix: [defensive control]
Limitations: [what was not tested]
```

6. Stop conditions

- The asset is not explicitly in scope or the rules are unclear.
- A test would access, modify, or disclose another real user's data.
- A test could disrupt service, cause persistent changes, or create costs.
- You encounter secrets, personal data, or an unexpected production system. Stop and follow the program's disclosure guidance.
- A platform rule or local law prohibits the activity.

Trusted beginner resources

- PortSwigger Web Security Academy - free interactive labs

- OWASP Web Security Testing Guide
- OWASP Juice Shop project
- HackerOne: defining scope
- Bugcrowd: reviewing program briefs

Before you submit: final review

- The exact asset and activity were explicitly in scope at the time of testing.
- The report describes observed behaviour, not a guess based only on a tool warning.
- Reproduction uses only approved training or test accounts and has no destructive side effects.
- Evidence is minimal and redacted; no passwords, tokens, or other users' data are included.
- Impact is realistic, the suggested fix matches the cause, and limitations are stated.

A few words to know

Term	Plain-language meaning
Scope	The exact assets and activities the owner authorises.
Reproducible	Another authorised tester can follow the steps and observe the same result.
Impact	The realistic harm the issue could cause, supported by evidence.
Disclosure	How a finding is reported and what may be shared publicly.

REALITY CHECK Bug bounty is competitive. Learning, finding a valid issue, and receiving a reward are separate outcomes. A clear authorised lab portfolio is worthwhile even if a programme offers no reward.

A real lab for these notes

Topic: Scope, authorization, safe testing, evidence, and clear reports. These pages connect the existing OpenCyberPath notes to specific provider-hosted exercises. The card label is intentionally generic; the exact task name and direct URL are listed here so you can open the right exercise instead of a whole catalogue.

Concept checkpoints

- Program scope names the allowed assets, test types, exclusions, and rate limits.
- Do not infer permission from a hostname, public IP, or successful response.
- Collect minimal evidence needed to reproduce a finding; avoid personal data and destructive actions.
- Separate reproduction, impact, and mitigation. Never claim access beyond what you actually observed.

Lab: Unprotected admin functionality *[PortSwigger Web Security Academy]*
https://portswigger.net/web-security/access-control/lab-unprotected-admin-functionality
Access note: Use the page's own Access the lab button to create an isolated practice instance. Never test a real website based on this example.

Lab: SQL injection - retrieve hidden data *[PortSwigger Web Security Academy]*
https://portswigger.net/web-security/sql-injection/lab-retrieve-hidden-data
Access note: A deliberately vulnerable training instance is launched from the page. The exercise is scoped to the generated lab host.

ACCESS CHECK Open the exact task page and verify the current login, account tier, VM launch button, free/paid terms, and any cost notes before starting. A provider may change availability or retire a challenge.

Before you click Start

- Write down the provider's target/scope and use only the instance it creates for you.
- Keep this notebook beside the exercise. Use its original course data before trying commands on a local machine.
- Capture timestamps, filenames, row/packet IDs, and a short explanation; redact credentials, tokens, and personal data.
- If a page says retired, premium, or downloadable-only, use the alternative listed in the access note or return to the matching interactive lab above.

Commands you can explain

The host name shown is a placeholder. Replace it only with the generated host for the lab you intentionally launched. Never run these requests against a third-party website.

First download the matching sample file(s) from the existing card when a command names one. Keep the data in a practice folder and never execute strings stored in evidence.

HTTP / INSPECT YOUR GENERATED LAB ONLY

```
curl -i "https://YOUR-LAB-ID.web-security-academy.net/robots.txt"
```

Replace YOUR-LAB-ID with the exact host generated by the assigned Academy instance; this is not a command for any other host.

HTTP / INSPECT RESPONSE HEADERS

```
curl -sS -D - "https://YOUR-LAB-ID.web-security-academy.net/" -o /dev/null
```

Use the generated training host. Headers help describe status and controls; they do not prove a vulnerability alone.

POWERSHELL EQUIVALENT

```
curl.exe -i "https://YOUR-LAB-ID.web-security-academy.net/robots.txt"
```

Use curl.exe to avoid PowerShell's historical curl alias differences; substitute only the lab ID shown by the provider.

SCOPE RECORD / COMMAND LINE

```
git status --short
```

If keeping a local report or notebook in a Git repository, record only your own documentation changes; do not commit credentials or captured sensitive data.

COMMAND HABIT Check the current working directory, shell, target, and expected effect before pressing Enter. Save the exact command and the output that mattered.

Guided lab run

- 1. Before opening tools, write the program name, exact allowed asset, allowed techniques, out-of-scope actions, and stop conditions.
- 2. Launch the exact Academy lab from the provider page; copy the generated lab hostname and verify it belongs to your current instance.
- 3. Use the lab's instructions and minimal requests to observe the specified weakness.
- 4. Record request/response evidence with tokens, session cookies, and personal data removed.
- 5. Write a report with title, scope, prerequisites, numbered reproduction steps, actual result, expected result, impact, mitigation, and limits.

Evidence checklist

- [ ] What exact lab instance or sample file was used?
- [ ] Which command/query and time range produced the observation?
- [ ] What is the directly observed fact, separate from your inference?
- [ ] What benign/alternative explanation could fit?
- [ ] What extra record or control would help confirm the conclusion?

Check your understanding

Q Is testing a public site allowed because the same weakness appears in an Academy lab?

Think it through: No. Permission is specific to the assigned training instance or written program scope.

Q What is the minimum acceptable report evidence?

Think it through: Enough redacted request/response details and steps for the owner to reproduce the finding without exposing secrets.

Q What should you do if scope is unclear?

Think it through: Stop testing and ask the program owner or instructor for written clarification.

NOTEBOOK CLOSE-OUT Finish with: Observation / Evidence source / Why it matters / Alternative explanation / Next check / Limitation. A careful "not enough evidence yet" is a valid professional result.